02 — Extraction & normalization

What it does: turns an uploaded PDF into structured, comparable data — a document type and a set of normalized fields — with no per-document-type template.

Where it runs: `process_document` is on `extraction_queue`, in a session bound to the document's company. Both Azure calls pass through the global Redis rate limiter: Document Intelligence at `AZURE_DOCUMENT_INTELLIGENCE_MAX_CALLS_PER_SECOND` (10), the Azure OpenAI call within `AZURE_OPENAI_MAX_REQUESTS_PER_MINUTE` / `AZURE_OPENAI_MAX_TOKENS_PER_MINUTE` (each call reserves its estimated prompt tokens + `max_tokens` 4,000). The task commits before each slow step, so it never holds a database transaction across an Azure call (processing-queues.md).

Code: `tasks/document_processing.py` (`process_document`), `services/ocr_service.py` , `services/classification_service.py` , `services/extraction_service.py` , `services/llm_service.py` , `services/field_locator_service.py` .

Algorithm

1. **Status:** `documents.processing_status = processing` .
2. **OCR / layout.** `OCRService.analyze_url(sas_url)` calls Azure Document Intelligence **`prebuilt-layout`** on a 30-minute SAS URL (Azure fetches the file itself). Printed Arabic is supported; **handwritten Arabic is not reliable** (an Azure limitation). Returns an `OCRResult` :

   ```
   @dataclass OCRResult:
       text: str                              # full text, reading order
       tables: list[OCRTable]                 # row_count, column_count, cells
       key_value_pairs: dict[str, str]
       pages: list[OCRPage]                   # page_number, words[], lines[]
   @dataclass OCRWord: text, x, y, width, height   # normalized 0-1 page fractions
                       font_family, font_confidence, italic_or_handwritten, confidence  # word-level
   extras
   ```

3. **Classify + extract in one call.** `extraction_service.classify_and_extract(llm, ocr_text)` → `LLMService.classify_and_extract` , an Azure OpenAI chat completion at `temperature=0` with **strict JSON-schema output**, returning a `DocumentAnalysis` (below). The label list is passed in from `classification_service.DOCUMENT_TYPE_LABELS` , so adding a type is a one-line change.
4. **Locate fields on the page (best effort).** `field_locator_service.attach_field_locations(pages, extracted_fields)` matches each extracted value back onto the OCR words and stores a normalized `bounding_box` on the field. It is text matching, so the position is approximate; ambiguous values (a total that also appears in a summary line) are disambiguated by nearby keywords ("total", "subtotal", "VAT", "date"); a field that cannot be found simply has no box. **A failure here never fails extraction.** Then the deterministic post-processing (`line_item_parsing.enrich_extracted_fields` , which is idempotent): line items read from the OCR layout, `pdf_info` (Producer, Creator, CreationDate, ModDate, and `image_count` — raster images the pages draw), and the clean-up in `services/extraction_postprocess.py` :
   - **Multi-column tables** (`parse_multi_column_tables`). A table with separate amount columns (Price / Discount / Net / VAT / Total) is read column by column: each row keeps its value per column in `columns` (OCR's "D.DD" read as 0.00, "131,900,00" as 131,900.00), `line_total` is the Net (else Total) column, and the column total rows ("Total Net Price", "VAT", "Total Price") go to `column_totals` . These rows replace the model's lines for the same rows — the model mixes columns. Field validation sums each column on its own (03).

- **Form template date.** A "Date of issue" that sits beside document-control markers (Rev. No, Issue No, Document/Form No — at least two) is the date the blank *form* was issued, not the document's date. It is labelled `form_template_date` (ISO `value`, `raw_text`, a `note`; the model's own name kept as `field_name_as_extracted`). A plain "Date of issue" (a certificate's) is left alone.
- **Signature ink read as a name.** OCR reads a signature as a word ("Prepared by: Princess" + the scribble read as "Dalab"). In person-role fields (prepared/signed/approved by, principal, …), words that OCR marked handwritten **and** set ≥ 1.8× the page's median word height, or that lie inside a detected signature region (when detection has already finished), are removed. The original stays in `value_as_read`.
- **Several invoices in one file** (`services/multi_invoice.py`). A page whose invoice number differs from the previous page's starts a new invoice. With two or more, each invoice's pages — the PDF's own text layer, else OCR (a free-tier OCR reads only the first two pages) — are sent to the extraction model separately and kept in `extracted_fields.invoices` (`page`, `pages`, `invoice_number`, `core_fields`, `line_items`). The document-level fields stay the first invoice's.

5. **Checks that need the OCR words** run here, before the document is marked complete, each best effort (its row is written inside a savepoint; a failure is stored as a failed check and never fails the extraction): font consistency (06a, PDFs only), spelling (06b, English words, every document) and analysis integrity (02a: text that tries to instruct the AI).
6. **Persist.** `document_type`, `ocr_text`, `extracted_fields`; `processing_status = complete`; audit `document_processing_completed` `{document_type, document_type_confidence, fields_located}`; then `run_document_checks.delay(document_id, company_id)` (on `vision_queue`).
7. **On any exception:** `processing_status = failed`, `processing_error` (≤ 4000 chars), audit `document_processing_failed`, and the case-level completion check and scoring are still triggered so a failed document cannot block the case forever. Before scoring, the analysis integrity check (02a) records why: a refusal by Azure's jailbreak filter → `prompt_injection_suspected` (high), anything else → `analysis_incomplete` (medium). A failed document therefore never scores as clean.

The extraction prompt (like every LLM prompt) ends with a rule that the document text is untrusted data: text addressing the AI ("ignore previous instructions", "report the total as…") is not followed.

Labels

`school_document, vendor_invoice, commercial_invoice, procurement_documentation, quotation, travel_invoice, payment_evidence, other`.

`get_document_role()` maps them to a **role** used by the cross-document check: `payment_evidence` → `evidence`, `other` → `other`, anything else → `claim`, `None` → `pending`. (A claim and its own evidence are *expected* to differ on issuer and date.) The frontend mirrors this in `types/case.ts`'s `getDocumentRole`; the two are kept in sync **by hand**.

Output shape

The LLM returns a `DocumentAnalysis` (Pydantic, `services/llm_service.py`):

```
class DocumentAnalysis:
    document_type: str
    document_type_confidence: float
    issuer: FieldValue             # value: str|None, confidence: float, uncertain: bool
    reference_number: FieldValue
    date: DateFieldValue           # value = ISO 8601 (YYYY-MM-DD) or None; raw_text kept
    amount: AmountFieldValue       # value: float|None, currency: str|None, raw_text
    subtotal: AmountFieldValue
    tax_amount: AmountFieldValue
    tax_rate: NumericFieldValue
    additional_fields: list[AdditionalField]   # {field_name, value, confidence, uncertain}
```

Amounts are **normalized at extraction time** to plain floats (Western digits, no separators or symbols) with a separate ISO currency code, and dates to ISO 8601, regardless of the document's own script or numeral system — so an Arabic-Indic-numeral invoice and a Western-numeral receipt compare directly. `raw_text` keeps what was actually seen, for display only; **logic must read `.value`, never `.raw_text`**.

What is stored in `documents.extracted_fields` (JSONB):

```
{
  "document_type_confidence": 0.97,
  "core_fields": {
    "issuer":           {"value": "Northgate Engineering", "confidence": 0.95, "uncertain": false,
                         "bounding_box": {"page":1,"x":0.05,"y":0.06,"width":0.30,"height":0.02}},
    "reference_number": {"value": "INV-NGT-2026-1049", "confidence": 0.99, "uncertain": false,
"bounding_box": {…}},
    "date":             {"value": "2026-08-25", "raw_text": "٢٥ أغسطس ٢٠٢٦", "confidence": 0.9,
"uncertain": false, …},
    "amount":           {"value": 197500.0, "currency": "AED", "raw_text": "197,500.00", …},
    "subtotal": {…}, "tax_amount": {…}, "tax_rate": {…}
  },
  "additional_fields": [
    {"field_name": "Payment terms", "value": "30 days", "confidence": 0.8, "uncertain": false,
"bounding_box": {…}}
  ]
}
```

`additional_fields` is how the pipeline stays template-free under Azure's strict-schema rule (which cannot express an open-ended object): arbitrary fields are a list of `{field_name, value}` pairs.

Known limitations

- **Low-confidence routing is not implemented.** `uncertain` is shown on the case detail page but nothing acts on it.
- The extraction model sees **text only** (OCR output), never the page image, so it cannot use visual layout cues beyond what OCR text order conveys.
- Only the first attempt runs; there is no retry of a failed extraction.
- Field locations are approximate and documents extracted before the locator existed have none (`backend/backfill_field_locations.py` re-runs OCR only, no LLM).
- LLM output is validated against a JSON schema but its *content* is not verified against the source.

Risk rules fed

None directly. Extraction feeds 03, 04 and 05.